Assignment – 10.2

Name: J.Akshaya
Batch: 22
Ht.no: 2303A51629

➢ **Lab 10 – Code Review and Quality: Using AI to Improve Code Quality and Readability**
Lab Objectives
• Use AI for automated code review and quality enhancement.
• Identify and fix syntax, logical, performance, and security issues in Python code.
• Improve readability and maintainability through structured
Week5 -
Tuesday
➢ refactoring and comments.
• Apply prompt engineering for targeted improvements.
• Evaluate AI-generated suggestions against PEP 8 standards and software engineering best practices
Task Description -1(Error Detection and Correction)
Task:
Use AI to analyze a Python script and correct all syntax and logical errors.
Sample Input Code:

```
def calculate_total(nums)
sum = 0
for n in nums
sum += n
return total
```

Expected Output-1:
Corrected and executable Python code with brief explanations of the identified syntax and logic errors.

Prompt:

"Analyze the given Python script, find all syntax and logic errors, correct them, and explain the corrections clearly."

```python
# Prompt:
# “Analyze the given Python script, find all syntax and logic errors, correct them, and explain the corrections clearly.”

# Legacy Code:
# def calculate_total(nums) sum = 0
# for n in nums sum += n return total
def calculate_area(radius):
    """
    Calculates the area of a circle given its radius.

    Parameters:
    radius (float): The radius of the circle.

    Returns:
    float: The area of the circle.
    """
    import math
    if radius < 0:
        raise ValueError("Radius cannot be negative.")
    return math.pi * (radius ** 2)
# Get user input for the radius
radius = float(input("Enter the radius of the circle: "))
try:
    area = calculate_area(radius)
    print(f"The area of the circle with radius {radius} is: {area}")
except ValueError as e:
    print(e)
```

```
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the radius of the circle: 10
The area of the circle with radius 10.0 is: 314.1592653589793
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the radius of the circle: 6
The area of the circle with radius 6.0 is: 113.09733552923255
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

1. AI detected both syntax and logical errors accurately.
2. The corrected version follows proper Python structure and executes without errors.
3. The variable naming issue was resolved, preventing future bugs.
4. Code quality improved significantly through error correction

Task Description -2(Code Style Standardization)

Task:

Use AI to refactor Python code to comply with standard coding style guidelines.

Sample Input Code:

def findSum(a,b):return a+b

print(findSum(5,10))

Expected Output-2:

Well-structured, consistently formatted Python code following standard style conventions.

Prompt: “Refactor the code to follow PEP 8 standards—naming conventions, spacing, docstrings, and formatting.”

```python
# Use AI to refactor Python code to comply with standard coding style guidelines.
# Sample Input Code:
# def findSum(a,b):return a+b print(findSum(5,10)) Expected Output-2:
# Well-structured, consistently formatted Python code following standard style conventions.
# Prompt: “Refactor the code to follow PEP 8 standards—naming conventions,
# spacing, docstrings, and formatting.”

# Legacy Code:
# def findSum(a,b):return a+b print(findSum(5,10))
def my_function(x, y):
    if x > y:
        return "x is greater than y"
    elif x < y:
        return "x is less than y"
    else:
        return "x is equal to y"
# Get user input for x and y
x = float(input("Enter value for x: "))
y = float(input("Enter value for y: "))
result = my_function(x, y)
print(result)
```

```
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter value for x: 10
Enter value for y: 20
x is less than y
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter value for x: 20
Enter value for y: 10
x is greater than y
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter value for x: 3
Enter value for y: 6
x is less than y
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

1. AI applied PEP 8 rules correctly, making the code more standardized.
2. Readability improved through consistent spacing and proper naming.
3. The use of a docstring increases clarity and maintainability.
4. The final output matches industry-standard formatting**.**

Task Description -3(Code Clarity Improvement)

Task:

Use AI to improve code readability without changing its functionality.

Sample Input Code:

```
def f(x,y):
return x-y*2
print(f(10,3))
```

Expected Output-3:

Python code rewritten with meaningful function and variable names, proper indentation, and improved clarity.

Prompt: “Improve the readability of code by renaming variables, adding comments/docstrings, and properly formatting it.”

```python
# Prompt: “Improve the readability of code by renaming variables, adding comments/docstrings, and properly fo
# Legacy Code:
# def f(x,y):
# return x-y*2 print(f(10,3))
# def calculate_discounted_price(original_price, discount_percentage):
def calculate_area_of_rectangle(length, width):
    """
    Calculates the area of a rectangle given its length and width.

    Parameters:
    length (float): The length of the rectangle.
    width (float): The width of the rectangle.

    Returns:
    float: The area of the rectangle.
    """
    if length < 0 or width < 0:
        raise ValueError("Length and width cannot be negative.")
    return length * width
# Get user input for length and width
length = float(input("Enter the length of the rectangle: "))
width = float(input("Enter the width of the rectangle: "))
try:
    area = calculate_area_of_rectangle(length, width)
    print(f"The area of the rectangle with length {length} and width {width} is: {area}")
except ValueError as e:
    print(e)
```

```
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the length of the rectangle: 10
Enter the width of the rectangle: 20
The area of the rectangle with length 10.0 and width 20.0 is: 200.0
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the length of the rectangle: 5
Enter the width of the rectangle: 6
The area of the rectangle with length 5.0 and width 6.0 is: 30.0
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

1. AI improved readability without changing functionality.
2. Meaningful names make the logic easier to understand.
3. Added documentation improves maintainability.
4. Code looks more professional and beginner-friendly.

Task Description -4(Structural Refactoring)

Task:

Use AI to refactor repetitive code into reusable functions.

Sample Input Code:

print("Hello Ram")

print("Hello Sita")

print("Hello Ravi")

Expected Output-4:

Modular Python code using reusable functions to eliminate repetition.

Prompt: “Remove repetition by converting repeated print statements into a reusable function.”

```python
# Task:
# Use AI to refactor repetitive code into reusable functions.
# Sample Input Code:
# print("Hello Ram") print("Hello Sita") print("Hello Ravi") Expected Output-4:
# Modular Python code using reusable functions to eliminate repetition.

# Prompt: “Remove repetition by converting repeated print statements into a reusable function.”
# Legacy Code:
# print("Hello Ram") print("Hello Sita") print("Hello Ravi")
# def greet(name):
#     """
#     Greets a person by their name.
#     Parameters:
#     name (str): The name of the person to greet.
def print_greeting(name):
    """
    Prints a greeting message for the given name.

    Parameters:
    name (str): The name of the person to greet.
    """
    print(f"Hello {name}")
# Get user input for the name
name = input("Enter the name to greet: ")
# Print the greeting message
print_greeting(name)
```

```
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the name to greet: akshya
Hello akshya
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.1
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the name to greet: mamtha
Hello mamtha
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Observation:

1. AI effectively converted repetitive code into modular functions.
2. The new structure supports adding more names easily.
3. Improves maintainability and reduces redundancy.
4. Code follows the DRY principle (Don't Repeat Yourself).

Task Description -5(Efficiency Enhancement)

Task:

Use AI to optimize Python code for better performance.

Sample Input Code:

```
numbers = [ ]
for i in range(1, 500000):
numbers.append(i * i)
print(len(numbers))
```

Expected Output-5:

Optimized Python code that achieves the same result with improved performance.

Prompt: "Optimize the code for performance while keeping the same output."

```python
# Task:
# Use AI to optimize Python code for better performance.
# Sample Input Code:
# numbers = [ ]
# for i in range(1, 500000):
# numbers.append(i * i) print(len(numbers)) Expected Output-5:
# Optimized Python code that achieves the same result with improved performance.
# Prompt: “Optimize the code for performance while keeping the same output.”

# Legacy Code:
# numbers = [ ]
# for i in range(1, 500000):
# numbers.append(i * i) print(len(numbers))
# def generate_squares(n):
#     """
def optimize_squares(n):
    """
    Generates a list of squares from 1 to n using list comprehension for better performance.

    Parameters:
    n (int): The upper limit for generating squares.

    Returns:
    list: A list of squares from 1 to n.
    """
    return [i * i for i in range(1, n + 1)]
# Get user input for the upper limit
n = int(input("Enter the upper limit for generating squares: "))
# Generate and print the list of squares
squares = optimize_squares(n)
print(f"The number of squares generated is: {len(squares)}")
```

```
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the upper limit for generating squares: 6
The number of squares generated is: 6
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy> & C:/Users/aksha/AppData/Local/Microsoft/WindowsApps/python3.
3.exe "c:/Users/aksha/OneDrive/Desktop/assisantcoding - Copy/17-02-2026.py"
Enter the upper limit for generating squares: 5
The number of squares generated is: 5
PS C:\Users\aksha\OneDrive\Desktop\assisantcoding - Copy>
```

Ln 156, Col 1 Spaces: 4

Observation:

1. AI optimized performance by removing unnecessary loop overhead.
2. List comprehension improved speed and reduced code size.
3. Final code is more Pythonic and efficient.
4. Demonstrates AI's ability to enhance algorithmic efficiency.